الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الخامس: مكتبة Matplotlib

الفصول ١ - ٦: من التعريف بالمكتبة حتى الدوال الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة 4
الفصل الأول: التعريف بمكتبة Matplotlib 5
١.١ نشأتها والمشكلة التي حلّتها 5
١.٢ الاستخدامات والشركات 5
١.٣ المميزات والعيوب 5
١.٤ متى تستخدمها ومتى لا 5
١.٥ أشهر البدائل 5
ملخص الفصل 6
الفصل الثاني: التثبيت والإعداد 7
٢.١ أفضل أسلوب استيراد 7
٢.٢ Backends 7
أفضل الممارسات 7
الفصل الثالث: فلسفة التصميم 8
٣.١ التسلسل الهرمي: Figure وAxes 8
٣.٢ واجهتان: pyplot مقابل Object-Oriented 8
٣.٣ Artist Hierarchy 8
٣.٤ نموذج التنفيذ: State Machine خلف الكواليس 8
ملخص الفصل 9
الفصل الرابع: المفاهيم الأساسية 10
٤.١ إنشاء Figure وAxes 10
٤.٢ العناصر البصرية الأساسية 10
٤.٣ الأنماط والألوان (Styling) 10
٤.٤ الحفظ بدقة عالية 10
الأخطاء الشائعة 10
الفصل الخامس: الكائنات وواجهات البرمجة 12
٥.١ أهم طرق Axes 12
٥.٢ Method Chaining المحدود في Matplotlib 12
٥.٣ نموذج التنفيذ: Immediate Mode مع تحديث مؤجل للعرض 12
أفضل الممارسات 12
الفصل السادس: أهم الدوال 13
٦.١ الرسوم الأساسية 13
٦.٢ الرسوم الفرعية (Subplots) 13

٦.٣ الخرائط الحرارية (Heatmaps) عبر imshow 13
٦.٤ الرسوم ثلاثية الأبعاد 13
٦.٥ التعليقات التوضيحية (Annotations) 13
٦.٦ الرسوم المتحركة (Animation) 13
الأخطاء الشائعة 14
الأسئلة الشائعة 14
تمارين عملية 14
حلول مقترحة (تلميحات) 14
مراجع وروابط رسمية 15
خاتمة الجزء الأول من Matplotlib 15

مقدمة

بعد أن أتقنّا معالجة البيانات (NumPy) وتنظيمها الجدولي (Pandas)، ننتقل الآن لخطوة لا تقل أهمية: تصويرها بصريًا. Matplotlib هي المكتبة الأساسية والأقدم للرسم البياني في بايثون، وهي الأساس الذي بُنيت فوقه مكتبات أخرى كثيرة (مثل Seaborn) وتتكامل معه مكتبات مثل Pandas مباشرة عبر df.plot().

الفصل الأول: التعريف بمكتبة Matplotlib

١.١ نشأتها والمشكلة التي حلّتها

طوّر John D. Hunter مكتبة Matplotlib في أوائل العقد الأول من الألفية الثانية، وكان حينها باحثًا في علم الأعصاب يستخدم MATLAB لتحليل ورسم إشارات دماغية، لكنه أراد بديلاً مفتوح المصدر بالكامل يعمل داخل بايثون. الاسم Matplotlib مُستوحى مباشرة من MATLAB (MATrix Plotting Library)، وواجهته الأولى (pyplot) صُممت عمدًا لتُحاكي أسلوب أوامر الرسم في MATLAB لتسهيل الانتقال على المستخدمين القادمين منه.

المشكلة التي حلّتها: توفير أداة رسم بياني علمية الدقة (Publication-quality)، قابلة للتخصيص الكامل برمجيًا، تتكامل بعمق مع NumPy، وتعمل عبر منصات وبيئات إخراج متعددة (شاشة تفاعلية، ملفات PNG وPDF وSVG، دفاتر Jupyter).

١.٢ الاستخدامات والشركات

- الرسم الاستكشافي السريع أثناء تحليل البيانات في Jupyter.
- إنتاج رسوم بيانية جاهزة للنشر العلمي (Publication-quality figures) في الأبحاث الأكاديمية.
- لوحات المراقبة والتقارير الآلية في الأنظمة الإنتاجية.
- الأساس البصري الذي تعتمد عليه Seaborn وPandas.plot() وأدوات أخرى كثيرة.

١.٣ المميزات والعيوب

الجانب	التفاصيل
مميزات	تحكم دقيق جدًا في كل عنصر بصري، نضج واستقرار طويل الأمد، تكامل عميق مع NumPy وPandas، مخرجات عالية الجودة لكل الصيغ
عيوب	واجهة pyplot الافتراضية أقل حداثة بصريًا من أدوات مثل Plotly، الرسوم التفاعلية تتطلب جهدًا إضافيًا (backends خاصة)، صياغة برمجية أطول أحيانًا لتحقيق تصميم بسيط
حدود	لا تدعم التفاعلية الغنية (Zoom, Hover, Filter) بشكل مدمج كما تفعل Plotly أصلاً

١.٤ متى تستخدمها ومتى لا

القرار	الحالة
Matplotlib	رسوم ثابتة عالية الدقة للنشر أو التقارير المطبوعة
Plotly أو Bokeh	رسوم تفاعلية للويب أو لوحات معلومات حية
Seaborn (المبنية فوق Matplotlib)	رسوم إحصائية سريعة بأقل كود ممكن مع تصميم جميل افتراضيًا
Matplotlib	تحكم برمجي دقيق جدًا في كل بكسل وعنصر بصري

١.٥ أشهر البدائل

- Seaborn: واجهة إحصائية عالية المستوى فوق Matplotlib، تصميم جميل افتراضيًا بأقل كود.
- Plotly: رسوم تفاعلية بالكامل، مناسبة لتطبيقات الويب ولوحات Dash.
- Bokeh: مشابهة لـ Plotly، تركّز على التفاعلية في متصفح الويب.
- Altair: مبنية على فلسفة Grammar of Graphics التصريحية (Declarative).

ملخص الفصل

- Matplotlib وُلدت كبديل مفتوح المصدر لـ MATLAB في الرسم البياني العلمي داخل بايثون.
- قوتها في التحكم الدقيق وجودة المخرجات؛ ضعفها في التفاعلية المدمجة مقارنة بـ Plotly.
- هي الأساس الذي تُبنى عليه Seaborn وPandas.plot().

الفصل الثاني: التثبيت والإعداد

```
pip install matplotlib
conda install matplotlib
```

٢.١ أفضل أسلوب استيراد

```
import matplotlib.pyplot as plt
import numpy as np
```

٢.٢ Backends

الـ Backend هو المحرك المسؤول عن كيفية عرض أو حفظ الرسم: بعضها تفاعلي (يُظهر نافذة رسم حية)، وبعضها غير تفاعلي (يُنتج ملفات فقط، مناسب للخوادم بلا واجهة رسومية).

```
import matplotlib
matplotlib.use('Agg')   # backend غير تفاعلي، مثالي للسكربتات والخوادم

# لعرض الرسوم كصور ثابتة مضمّنة داخل Jupyter
%matplotlib inline

# لعرض رسوم تفاعلية قابلة للتكبير داخل Jupyter
%matplotlib widget
```

أفضل الممارسات

- استخدم 'backend='Agg في السكربتات التي تعمل على خوادم بلا واجهة رسومية لتفادي أخطاء العرض.
- استخدم %matplotlib inline في دفاتر Jupyter الاستكشافية العادية.

الفصل الثالث: فلسفة التصميم

٣.١ التسلسل الهرمي: Figure وAxes

الفكرة المحورية في تصميم Matplotlib هي التسلسل الهرمي بين كائنين رئيسيين: Figure هو "اللوحة" أو النافذة الكاملة التي تحتوي كل شيء، وAxes هو "منطقة الرسم" الفعلية داخل تلك اللوحة (وليس نفس معنى axis أي المحور الفردي — وهذا الالتباس اللغوي مصدر ارتباك شائع للمبتدئين). يمكن لـ Figure واحد أن يحتوي عدة كائنات Axes (رسوم فرعية متعددة Subplots).

الكائن	الدور
Figure	اللوحة الكاملة/النافذة، تحتوي كل العناصر
Axes	منطقة رسم واحدة (قد يكون هناك عدة Axes داخل Figure واحد)
Axis	محور فردي (X أو Y) داخل Axes معيّن، مسؤول عن الحدود والعلامات (Ticks)
Artist	كل عنصر مرئي فعليًا (خط، نص، شكل) يُرسم داخل Axes

٣.٢ واجهتان: pyplot مقابل Object-Oriented

صُممت Matplotlib بواجهتين متوازيتين عن قصد: واجهة pyplot الإجرائية (State-based) المستوحاة من MATLAB وسهلة للاستخدام السريع، وواجهة كائنية التوجه (Object-Oriented) تمنحك تحكمًا صريحًا وكاملاً عبر كائنات Figure وAxes مباشرة. التوثيق الرسمي والمشاريع الاحترافية توصي بالواجهة الكائنية لأي كود يتجاوز رسمًا سريعًا استكشافيًا واحدًا، لأنها أوضح نية وأسهل صيانة عند وجود عدة رسوم فرعية.

```
# أسلوب pyplot (الإجرائي) سريع لكن أقل وضوحًا في الرسوم المعقدة
plt.plot([1, 2, 3], [4, 5, 6])
plt.title('عنوان')
plt.show()

# الأسلوب الكائني الموصى به احترافيًا
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [4, 5, 6])
ax.set_title('عنوان')
plt.show()
```

٣.٣ Artist Hierarchy

كل عنصر مرئي في Matplotlib — من الخط إلى النص إلى العلامة — هو كائن Artist يرث من فئة أساسية موحّدة، مما يمنح تناسقًا في كيفية التحكم بالخصائص (لون، سماكة، شفافية) عبر كل أنواع الرسوم. هذا التصميم الكائني العميق يفسّر لماذا يمكنك حفظ أي كائن رسم في متغير (كخط أو نقاط تشتت) وتعديل خصائصه لاحقًا بعد الرسم الأولي.

٣.٤ نموذج التنفيذ: State Machine خلف الكواليس

واجهة pyplot تعمل كآلة حالة (State Machine): تتذكّر آخر Figure وAxes 'نشطين' وتوجّه أي أمر رسم تالٍ (plt.plot, plt.title) إليهما تلقائيًا دون أن تحدد أنت أيهما صراحة. هذا مريح للاستخدام السريع لكنه مصدر أخطاء صامتة عند التعامل مع رسوم فرعية متعددة، وهو السبب الرئيسي وراء تفضيل الواجهة الكائنية الصريحة في الكود الاحترافي.

ملخص الفصل

- التسلسل الهرمي المحوري: Figure يحتوي واحدًا أو أكثر من Axes، وAxes يحتوي Axis وArtists.
- واجهتان متوازيتان: pyplot الإجرائية السريعة، والكائنية الصريحة الموصى بها للكود الاحترافي.
- pyplot تعمل كآلة حالة تتذكر آخر Figure/Axes نشطين، مما قد يسبب التباسًا في الرسوم المعقدة.

الفصل الرابع: المفاهيم الأساسية

٤.١ إنشاء Figure وAxes

```
fig, ax = plt.subplots(figsize=(8, 5))           # رسم واحد
fig, axes = plt.subplots(2, 2, figsize=(10, 8)) # شبكة 2×2 من الرسوم الفرعية
axes[0, 0].plot(...)
axes[0, 1].bar(...)
```

٤.٢ العناصر البصرية الأساسية

- Title: عنوان الرسم عبر ax.set_title().
- Labels: تسميات المحاور عبر ax.set_xlabel() وax.set_ylabel().
- Legend: مفتاح الرسم عبر ax.legend()، يعتمد على معامل label في كل عنصر مرسوم.
- Ticks: علامات القياس على المحاور، قابلة للتخصيص الكامل موقعًا ونصًا.
- Grid: خطوط الشبكة الخلفية عبر ax.grid(True).

```
fig, ax = plt.subplots()
ax.plot(x, y1, label='المبيعات')
ax.plot(x, y2, label='التكلفة')
ax.set_title('أداء الربع الأول')
ax.set_xlabel('الشهر')
ax.set_ylabel('القيمة')
ax.legend()
ax.grid(True, alpha=0.3)
```

٤.٣ الأنماط والألوان (Styling)

```
ax.plot(x, y, color='#2E5395', linestyle='--', linewidth=2, marker='o')
plt.style.use('seaborn-v0_8')   # تطبيق نمط جاهز على كل الرسوم اللاحقة
```

Matplotlib توفّر أنماطًا جاهزة (Style Sheets) تُغيّر المظهر العام (الألوان، الخطوط، الخلفية) بسطر واحد، وهي بداية جيدة قبل التخصيص اليدوي التفصيلي.

٤.٤ الحفظ بدقة عالية

```
fig.savefig('chart.png', dpi=300, bbox_inches='tight')
fig.savefig('chart.svg')   # صيغة متجهية بلا فقدان جودة عند التكبير
```

dpi=300 هو المعيار الشائع لجودة نشر أكاديمي أو تقارير احترافية مطبوعة، وbbox_inches='tight' يزيل الهوامش الفارغة الزائدة تلقائيًا حول الرسم.

الأخطاء الشائعة

- نسيان plt.show() في السكربتات خارج Jupyter (رغم أن Jupyter نفسه غالبًا يعرض الرسم تلقائيًا في الخلية الأخيرة).
- الخلط بين Axes (منطقة الرسم) وAxis (محور واحد) بسبب تشابه الاسمين.

- استخدام واجهة pyplot الإجرائية في رسوم فرعية متعددة معقدة، مما يسبب رسمًا في المكان الخطأ بسبب آلة الحالة الضمنية.

الفصل الخامس: الكائنات وواجهات البرمجة

٥.١ أهم طرق Axes

الاستخدام	الطريقة
رسم خطي	ax.plot()
رسم نقاط تشتت	ax.scatter()
أعمدة عمودية/أفقية	ax.bar() / ax.barh
مدرّج تكراري (Histogram)	ax.hist()
عرض مصفوفة ٢D كصورة/خريطة حرارية	ax.imshow()
تحديد نطاق المحاور يدويًا	ax.set_xlim() / ax.set_ylim

٥.٢ Method Chaining المحدود في Matplotlib

على عكس Pandas، لا تدعم معظم طرق Axes التسلسل (Chaining) لأنها تُرجع كائنات Artist وليس Axes نفسه، لذلك الأسلوب الاصطلاحي هو استدعاء الطرق كل على سطر منفصل على نفس كائن ax.

٥.٣ نموذج التنفيذ: Immediate Mode مع تحديث مؤجل للعرض

الرسم في Matplotlib يُبنى تدريجيًا: كل استدعاء (plot, scatter, set_title) يضيف أو يعدّل حالة كائنات Artist داخليًا فورًا، لكن العرض الفعلي على الشاشة أو الحفظ إلى ملف يحدث فقط عند استدعاء plt.show() أو fig.savefig()، مما يسمح بتجميع تعديلات كثيرة قبل الإخراج النهائي.

أفضل الممارسات

- احتفظ دائمًا بمرجع fig وax الناتجين من plt.subplots() بدل الاعتماد على الحالة الضمنية لـ pyplot.
- استخدم tight_layout() أو constrained_layout=True عند وجود رسوم فرعية متعددة لتفادي تداخل العناوين.

الفصل السادس: أهم الدوال

٦.١ الرسوم الأساسية

```
x = np.linspace(0, 10, 100)
y = np.sin(x)

fig, ax = plt.subplots()
ax.plot(x, y)                                    # خطي
ax.scatter(x[::5], y[::5], color='red')          # نقاط تشتت
ax.bar(['A','B','C'], [10, 25, 15])              # أعمدة
ax.hist(np.random.randn(1000), bins=30)          # مدرّج تكراري
ax.pie([30, 45, 25], labels=['A','B','C'])       # دائري
```

٦.٢ الرسوم الفرعية (Subplots)

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].plot(x, np.sin(x))
axes[0].set_title('Sine')
axes[1].plot(x, np.cos(x))
axes[1].set_title('Cosine')
fig.tight_layout()
```

٦.٣ الخرائط الحرارية (Heatmaps) عبر imshow

```
matrix = np.random.rand(10, 10)
fig, ax = plt.subplots()
im = ax.imshow(matrix, cmap='viridis')
fig.colorbar(im, ax=ax)   # شريط الألوان التوضيحي
```

٦.٤ الرسوم ثلاثية الأبعاد

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.scatter(x, y, np.sin(x) * np.cos(y))
```

٦.٥ التعليقات التوضيحية (Annotations)

```
ax.annotate('أعلى نقطة', xy=(5, 1), xytext=(6, 1.2),
            arrowprops=dict(facecolor='black', shrink=0.05))
```

٦.٦ الرسوم المتحركة (Animation)

```
from matplotlib.animation import FuncAnimation

fig, ax = plt.subplots()
line, = ax.plot(x, np.sin(x))

def update(frame):
```

```
    line.set_ydata(np.sin(x + frame / 10))
    return line,

anim = FuncAnimation(fig, update, frames=100, interval=50)
```

FuncAnimation تعيد رسم عنصر Artist محدد (هنا line) في كل إطار بدل إعادة بناء الرسم بالكامل، وهو أسلوب أكفأ أداءً يُعرف بـ Blitting عند تفعيله صراحة.

الأخطاء الشائعة

- استخدام plt.plot() المباشر (الحالة الضمنية) بدل ax.plot() في رسوم فرعية متعددة.
- نسيان fig.colorbar() عند استخدام imshow، مما يترك الخريطة الحرارية بلا مرجع لقراءة القيم.
- استخدام plt.show() داخل حلقة رسوم متحركة يدوية بدل FuncAnimation المخصصة لهذا الغرض.

الأسئلة الشائعة

س: ما الفرق بين plt.plot وax.plot؟

ج: الأولى تستدعي آلة الحالة الضمنية لترسم على آخر Axes نشط، بينما الثانية ترسم صراحة على كائن Axes محدد؛ الثانية أوضح وأكثر أمانًا في الرسوم المعقدة.

قصده هنا ان الاولي بستخدم فاكشن ممكن انها اتعدل عليها قبل كده وانا بضيف قيم جديدة وانسي اغير كل الاعدادات علي عكس التانية اللي بتكون اوبجكت للكل رسمه يعني مفيش تعديل لرسمة بيأثر علي اوبجكت لرسمة تانيه

س: كيف أختار خريطة ألوان (Colormap) مناسبة؟

ج: استخدم خرائط متسلسلة (viridis, plasma) للبيانات الكمية المتصلة، وخرائط متباينة (coolwarm) للبيانات ذات قيمة وسطى مرجعية، وتجنّب jet لأنها مضلِّلة إدراكيًا.

تمارين عملية

١. ارسم رسمًا فرعيًا ٢×١: الرسم العلوي لدالة sin، والسفلي لدالة cos، بعناوين ومحاور مسمّاة.

٢. ارسم مدرجًا تكراريًا لـ ١٠٠٠ قيمة عشوائية من توزيع طبيعي، مع خط عمودي يوضّح موقع المتوسط عبر ax.axvline.

٣. احفظ أي رسم سابق بصيغة PNG بدقة ٣٠٠ وبصيغة SVG.

حلول مقترحة (تلميحات)

١. fig, axes = plt.subplots(2, 1); axes[0].plot(x, np.sin(x)); axes[1].plot(x, np.cos(x))

٢. ax.hist(data, bins=30); ax.axvline(data.mean(), color='red', linestyle='--')

```
import numpy as np
import matplotlib.pyplot as plt

# توليد البيانات وحساب المتوسط في سطرين
data = np.random.normal(0, 1, 1000)
mean_val = data.mean()

# الرسم والتنسيق بشكل مختصر
fig, ax = plt.subplots()
ax.hist(data, bins=30, color='skyblue', edgecolor='black')
ax.axvline(mean_val, color='red', linestyle='--', label=f'Mean:
{mean_val:.2f}')

# تجميع العناوين والمحاور في أمر واحد
ax.set(title='Histogram', xlabel='Value', ylabel='Frequency')
ax.legend()
plt.show()
```

٣. fig.savefig('out.png', dpi=300); fig.savefig('out.svg')

مراجع وروابط رسمية

- التوثيق الرسمي: https://matplotlib.org/stable/
- معرض الأمثلة الرسمي: https://matplotlib.org/stable/gallery/index.html
- الكود المصدري: https://github.com/matplotlib/matplotlib

خاتمة الجزء الأول من Matplotlib

غطّينا تاريخ المكتبة وفلسفتها القائمة على Figure وAxes، وأهم دوال الرسم والتخصيص. الجزء الثاني سيتناول أمثلة عملية متقدمة، الأداء، التصحيح، أفضل الممارسات، ومشاريع مصغّرة.